文章编号: 1000-5641(2025)05-0140-11

基于 DTA 的 GitHub 高星仓库活跃度评估方法

游明东,　彭佳恒,　韩凡宇,　王　伟

(华东师范大学 数据科学与工程学院, 上海　200062)

摘要: 以识别 GitHub 长期活跃高星仓库帮助开源社区构建和数字基础设施建设为背景, 提出了一种基于时间序列预测模型的 GitHub 高星仓库长期活跃度评估方法, 旨在解决识别仓库是否能够保持长期活跃的问题. 该方法首次引入开发者活跃周期作为关键特征, 用以提升仓库发展趋势预测的准确性. 通过对活动指标的时间序列数据进行建模与挖掘, 该方法提出了全新的活跃度计算公式 DTA (Development Trend-based Activity), 实现了对仓库活跃水平的准确量化评估. 设计并制作了一个时间粒度细、覆盖范围广的基准数据集, 并系统评估了多种预测模型的表现, 最终确定了适用于开源仓库活跃度预测的最优模型. 实验结果验证了所提方法的有效性, 能够准确预测仓库的长期活跃情况. 因此, 引入 DTA 对仓库活跃度进行评估, 能够帮助开源参与者识别长期活跃的仓库, 确定参与重心, 促进开源社区的构建和数字基础设施建设.

关键词: 开源软件仓库;　活跃度评估;　开源社区

中图分类号: TP391　　**文献标志码**: A　　**DOI**: 10.3969/j.issn.1000-5641.2025.05.013

A DTA based activity evaluation method for high star GitHub repositories

YOU Mingdong,　PENG Jiaheng,　HAN Fanyu,　WANG Wei

(*School of Data Science and Engineering, East China Normal University, Shanghai*　200062, *China*)

Abstract: In the context of identifying GitHub's long-term active, high-star repositories—critical for assisting the development of robust open-source communities and vital digital infrastructure—we propose a novel method for evaluating the long-term activity of these repositories. This method is firmly based on a time series prediction model, which excels in forecasting repository activity metrics rather than being specifically designed for this purpose. A key innovation of our method is the first-time use of the developer activity cycle as a pivotal feature. This improves the accuracy of predictions for repository development trends and provides a more nuanced understanding of project evolution. After meticulously modeling and mining the time series data of various activity indicators, we developed a new activity calculation formula: development trend-based activity (DTA). This formula allows a precise quantitative evaluation of a repository's true activity level. To rigorously validate our methodology, we designed and curated a comprehensive benchmark dataset with fine time granularity and broad coverage. Subsequently, we systematically evaluated the performance of multiple prediction models against this dataset, eventually identifying the best model for forecasting open-source repository activity. The experimental results conclusively demonstrate the effectiveness of our proposed method in accurately predicting the long-term activity of repositories. Consequently, using DTA to evaluate repository activity can enable open-source participants to effectively identify repositories poised for long-term engagement, strategically determine their participation focus, and thereby significantly promote the sustained development of open-source communities and critical digital infrastructure.

收稿日期: 2025-07-03

通信作者: 王　伟, 男, 教授, 博士生导师, 研究方向为计算教育学、开源数字生态学.
E-mail: wwang@dase.ecnu.edu.cn

中国知网　https://www.cnki.net

Keywords: open-source repository; activity assessment; open-source community

0 引　言

随着开源模式以公开、共享的特点被人们广泛接受和认可后, GitHub 作为世界上最大的开源软件平台, 其开源软件仓库 (简称仓库) 数量迎来了飞速增长. 根据 GitHub Octoverse Report 可知, 在 2024 年, 被创建的仓库数高达 10.8 亿个[1]. 这些仓库相互依赖, 共同构成了许多软件生态, 同时也成为了数字基础设施的一部分[2]. 但是, 只有其中的小部分仓库保持了活跃, 大量的仓库则由于核心开发者缺失或者缺乏长期发展规划等原因在创建后很快就销声匿迹, 无法保持长期维护, 这对计算机领域、开源社区构建和人们的日常生活都造成了很大影响[3]. 因此, 如何识别出能够保持长期活跃的仓库仍然是一个亟待解决的问题.

之前的研究已经调查了各种因素对仓库长期活跃程度的影响, 例如, 仓库的提交、问题、版本更新次数和拉取请求等信息[4–5]. 然而, 这些评估方法仅仅利用了历史信息而忽略了对仓库未来发展趋势的研究与利用[6–7]. 本文创新性地利用时间序列预测模型获取了仓库未来的发展趋势, 结合仓库未来发展趋势中的活动数据对其长期活跃程度进行了完整的评估.

为了研究对开源社区构建和数字基础设施有重大影响的仓库, 本文收集并制作了一份 GitHub 高星软件仓库的活动指标时间序列数据集, 同时引入了前沿的时间序列预测模型, 并结合对开源参与者行为模式的挖掘来预测仓库未来的发展趋势. 此外, 本文还设计了一个仓库活跃度评估公式 DTA (Development Trend-based Activity), 对发展趋势中的活动指标进行提炼, 以评估仓库在未来的活跃情况.

本文的主要贡献包括以下部分.

1) 设计并制作了一份 GitHub 高星软件仓库活动指标时间序列数据集. 该数据集时间粒度细, 覆盖范围广, 涵盖各类主题的仓库, 编程语言类型超 40 种, 开源许可证类型超 19 种, 总数据量超 45 万条, 可用于多种时间配置的时间序列预测任务研究.

2) 针对如何准确量化并预测开源仓库未来活跃度的挑战, 设计了一种仓库活跃度评估公式 DTA. 该公式综合考虑了仓库的代码及文本相关活动指标, 通过非线性函数控制指标边际效应, 能够对不同规模大小的仓库进行公平评估.

3) 针对提出的 DTA 评估公式进行了大量实验, 验证了其有效性. 其中, 利用 Pearson 相关系数和 Spearman 相关系数验证了 DTA 与活动指标的强正相关性, 并利用平均斜率验证了 DTA 的现实意义.

1 相关工作

1.1 开源仓库长期活跃相关活动指标

开源仓库对软件生态构建和数字基础设施建设具有十分重要的意义, 其合作共享的方式使仓库能够聚集大量优秀的开源社区开发者. 因此, 开发者们的活动、仓库管理者本身的属性及其所遵循的开源模式等都对仓库的长期活跃有着不同的影响. 在过去的研究中, 国内外学者们分析了许多不同的因素. 例如, 是否践行高质量的实践标准[8] (模块化设计、易于理解的使用文档); 是否资金充足, 附加值高[9–10]; 是否有足够的开发活动指标 (如问题和拉取请求)[4–5]. 这些因素中, 持续的开发活动是十分重要的信号, 如果不进行维护, 任何软件都会越来越无用[11]. 而对于持续活动指标的研究中, 有学者指出了代码相关活动和文本相关活动的重要性, 并通过模型无关解释器分析了其中指标对仓库活跃状况的影响[6]. 因此, 本文结合前人研究的结果, 确定了仓库活跃程度的关键持续活动指标.

中国知网 https://www.cnki.net

1.2 开源仓库长期活跃程度评估方式

仓库的长期活跃程度与其未来发展趋势中的活动指标数量息息相关. 而当前的评估方式一般仅利用历史信息对仓库的状态进行判断, 忽略了对未来发展趋势的利用. 在目前的仓库活跃程度研究中, Xiao 等[6] 利用 XGboost 模型结合仓库早期活动指标对仓库未来的状态进行了分类; Coelho 等[7] 利用随机森林模型对仓库两年的历史信息进行了分析, 用于模型对仓库状态进行分类; Xia 等[12] 通过实证研究分析了开源社区开发者们的行为模式及其与仓库长期活跃的关系, 提出了线性模型难以完全反映仓库活动因素的影响. 这些研究的重点都是对历史信息的利用, 是对项目状态的分类而不是预测. 本文采用时间序列预测模型, 结合开发者行为模式, 获取了仓库未来发展趋势, 以评估仓库的长期活跃程度.

时间序列预测模型方面, Transformer[13] 证明了其不同数据模型下的巨大潜力. 因此, 在前沿时间序列预测模型上, 学者们将 Transformer 引入时间序列预测任务. LogTrans 在原 Transformer 的基础上设计了一个卷积自注意力层, 降低了空间复杂度[14]. Autoformer 借鉴了传统时间序列分析方法和自相关的思想进行自回归预测[15]. Liu 等[16] 提出将模型输入转置, 使模型输入的令牌在语义上更加完整, 提高了预测准确性. PatchTST 利用分块的思想降低了计算复杂度, 同时携带了更多的历史信息, 在一众预测模型中取得了最好的预测效果[17]. 本文使用 PatchTST 模型对仓库未来发展趋势进行预测, 后文将通过实验证明其预测的准确性.

在具体的评估方式中, 前人主要利用线性模型对仓库活动指标进行分析, 这些方法忽略了仓库规模的差异, 也难以反映活动因素的实际影响. 本文提出的评估公式 DTA 能够公平地评估不同规模的仓库, 并且通过引入非线性函数控制活动指标的边际效应, 使其准确地反映仓库未来实际的活跃情况.

2 研究方法

本章将首先介绍本文的研究问题, 然后介绍数据集详细的制作方法, 之后对本文使用的模型和评估方法做出解释, 研究方法概述如图 1 所示. 在采集 GitHub 仓库的事件数据后, 先对原始数据进行清洗, 包括异常值处理, 数据平滑与放缩. 接着, 利用时间序列预测模型结合开发者活动周期对每个活动指标进行预测, 在获得仓库未来一段时间的活动指标后提取其中的平均值, 并结合 Zhao 等[18] 在实践中利用层次分析法赋予各活动指标不同的权重. 最后, 利用活跃度公式 DTA, 得到仓库的长期活跃程度评分.

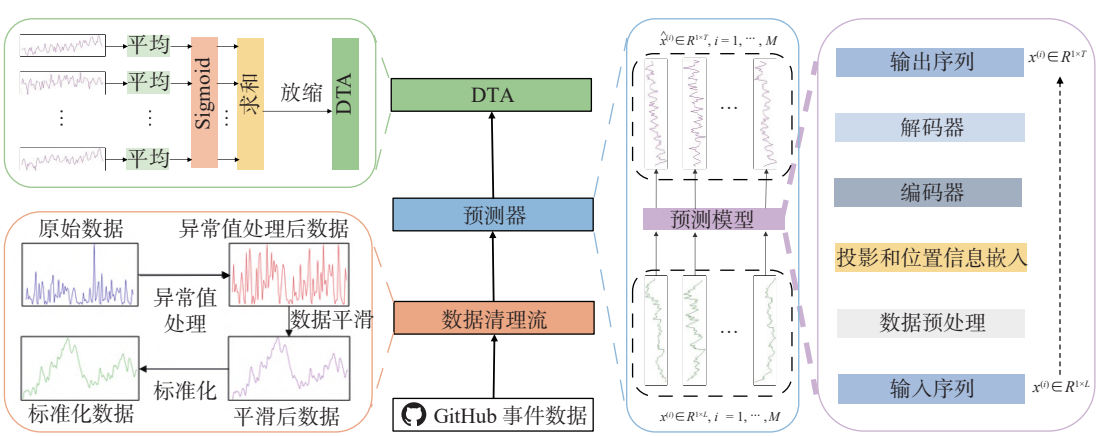


图 1 基于 DTA 的仓库长期活跃度评估方法框架

Fig. 1 Framework for long term activity evaluation of repository based on DTA

2.1 研究问题

本文主要聚焦于利用开源软件仓库未来的活动指标来评估其当前时间的活跃度, 特别地, 希望能

中国知网 https://www.cnki.net

够回答如下问题.

研究问题一 (RQ1): 如何准确地预测仓库的发展趋势? 仓库的发展趋势通过其活动指标体现, 但由于活动数据的非线性和偏态分布等特征, 直接使用时间序列预测模型进行预测的效果十分不理想, 本文将研究如何使用时间序列预测模型对仓库活动指标进行全面、准确的预测.

研究问题二 (RQ2): 如何评估一个仓库的活跃度? 仓库难以长期发展的问题对开源社区的构建、数字基础设施的建设和开源社区参与者造成了维护、选择等困扰, 本文将结合仓库的未来发展趋势研究如何评估仓库的活跃度并验证其准确性.

2.2　数据获取与预处理

本文利用 GHArchive 采集了 GitHub 平台 2021—2024 年的事件数据, 并将其存储于基于列存的 ClickHouse 云服务中, 以支持基于 SQL 的实时在线分析. 原始活动数据存在时间步空缺多、受人为因素干扰强、偏态分布明显以及周期性与非线性特征显著等问题, 难以直接用于时间序列预测建模. 图 2 展示了由于人为拆分导致 Homebrew/Legacy-homebrew 仓库活跃度骤降的案例, 图 3 则显示了 Significant-gravitas/AutoGPT 仓库因热度骤减而呈现偏态分布的情况. 为提升预测模型适用性, 本文对原始数据进行了系统清洗与重构, 构建了适用于预测任务的数据集.

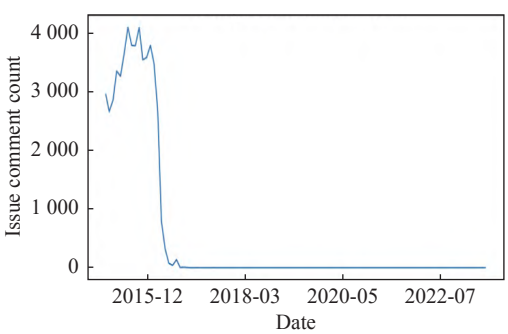


图 2　Homebrew 仓库 Issue comment 指标增长曲线

Fig. 2　Homebrew repository issue comment indicator growth curve

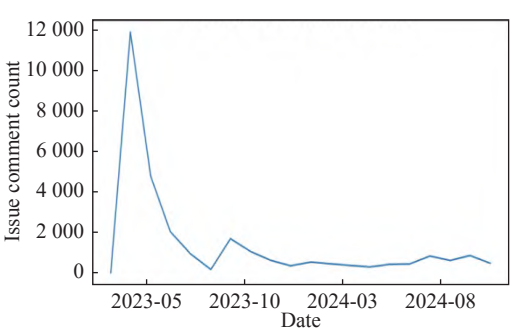


图 3　AutoGPT 仓库 Issue comment 活动指标增长曲线

Fig. 3　AutoGPT repository issue comment activity indicator growth curve

高星 (star) 仓库通常拥有更稳定的维护频率、较长的生命周期和更充分的活动数据, 更符合“长期活跃度”评估的目标, 具备更好的可建模性与可预测性. 本文选取了 GitHub 平台 2021—2023 年间 star 数排名前 1 000 的高星仓库, 并且从仓库的文本活动及代码活动方面进行活动指标选取. 在 GitHub 中, 用户通过 Issue 功能报告目前仓库中的问题, 例如漏洞报告、功能需求等, 并且可以通过 Comment 功能与仓库开发者或者其他用户进行文本交流. 另外, 用户们也通过 Pull request 功能对仓

中国知网　https://www.cnki.net

库进行代码贡献, 例如漏洞修复、新功能实现等. 本文最终确定了仓库的 5 个持续活动指标, 分别是 Open issue、Issue comment、Open pull request (OpenPR)、Pull request review comment (PRcomment)、Merge pull request (MergePR). 这 5 个活动指标能够全面地反映仓库的用户交流活跃情况和维护情况.

本文按照如下方式对数据进行筛选.

1) 剔除无 Issue 或 PR 相关活动指标的仓库. 移除因关闭相关功能而导致长期活跃度无法评估的仓库.

2) 剔除 fork 版本的仓库. 确保评估对象的独立性和代表性, 避免 fork 仓库数据与原仓库的耦合而对结果产生干扰.

筛选后, 本文收集了剩余仓库上述 5 个活动指标自 2021-01-01 至 2023-01-01 的每日增量时间序列数据. 由于事件驱动的记录方式, 各仓库序列长度不一, 同时针对原始数据的时间步不连续性和增量特性, 将空缺时间步的指标值设为 0. 此外, 为规避人为因素影响并克服数据的偏态分布、非线性、周期性等特点, 本文对数据集进行了清理.

1) 异常值处理. 利用 z-score 方法对数据集进行异常值处理. 使用$|z\text{-score}| = 2$ 作为异常处理的阈值, 阈值内涵盖 95% 的数据. 对超出阈值的异常值使用滚动窗口均值代替.

2) 数据平滑. 对异常值处理后的数据采用指数加权移动平均法进行平滑. 该方法对近期数据赋予了更高权重. 具体地, 结合开发者活动周期将平滑窗口大小设置为 28, 将开发者活动指标利用一个工作月的数据进行平滑, 有效抑制了时间序列中的异常波动.

3) 数据放缩. 利用最大最小值放缩法对数据平滑后的数据放缩至 [0, 1] 区间, 放缩后的值是其相较于历史极值的比例关系. 2.4 节会详细解释对放缩值的利用.

最后, 本文制作的数据集中涵盖 JavaScript、Python、C、Go 等超过 40 种编程语言, MIT、Apache 2.0、BSD 3-Clause 等 19 种开源许可证类别. 其中, 62.6% 的仓库拥有者为组织, 37.4% 为个人用户, 总数据量超 45 万条, 可用于多种时间配置下的时间序列预测任务研究.

2.3 时间序列预测模型

本文通过对前沿时间序列预测模型进行调研和实际预测效果评估, 发现 PatchTST (Patch Time Series Transformer) 模型相较于其他模型有着最低的预测误差 (4.66), 详细结果见后文. 因此, 最终选择 PatchTST 作为仓库活动指标的预测模型. 该模型能够在保证高预测准确性的同时, 显著降低计算资源消耗并有效捕捉局部历史信息.

PatchTST 是一种专门为长时间序列预测设计的基于 Transformer 模型的变体[17]. 该模型通过设计 patch 组件和通道独立组件来对传统的基于 Transformer 的时间序列预测模型进行改进, 实现对长时序数据的高效特征提取与时序依赖建模, 解决了传统时序模型在处理长序列时面临的计算复杂度高、全局依赖捕获能力不足等问题.

模型框架具体如图 4 所示, 多变量序列在输入 Transformer 主干网络之前通过通道独立组件分割为单变量序列 $x^{(i)} \in R^{1\times L}$. 其中, i 表示第 i 个变量, L 为序列长度. 对单变量序列归一化后利用 patch 模块将长序列分割为固定长度的子序列 $x^{(i)} \in R^{P\times N}$, 其中, P 为分块过程中滑动窗口大小, N 为分块数目, 其计算方式为

$$N = \left\lfloor \frac{L-P}{S} \right\rfloor + 2. \tag{1}$$

式 (1) 中: S 为滑动窗口移动过程中可重复窗口的大小. 该步骤通过降低序列长度维度来显著减少后续 Transformer 处理的计算量. 分块后的子序列经线性投影层映射至特征空间 $R^{D\times N}$, 其中, D 为编码器隐藏层维度. 将映射后的结果经 Transformer 编码器得到 $z^{(i)} \in R^{D\times N}$. 再经扁平化操作转化为一维

中国知网 https://www.cnki.net

向量, 聚合全局时序特征, 最后通过线性层映射至目标空间, 最终输出单变量时序序列预测结果 $\hat{x}^{(i)} \in R^{1\times T}$.

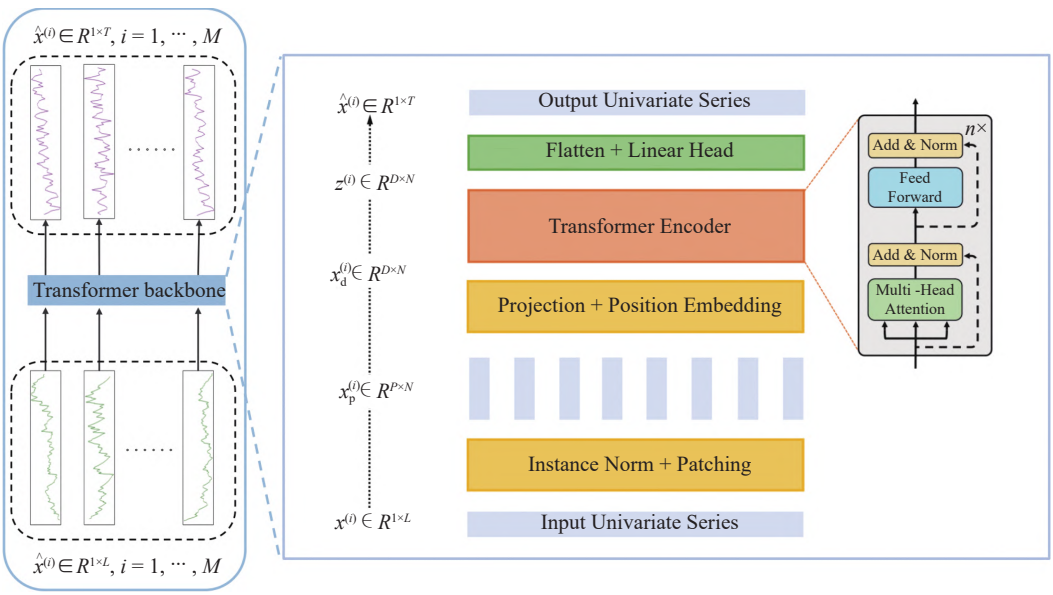


图 4　PatchTST 框架图

Fig. 4　PatchTST framework diagram

该模型的核心在于将原始时间序列数据切分成多个 patch 以及通道独立机制, 这些组件为基于 Transformer 的时序模型带来了以下优势.

1) 降低计算复杂度: 通过分块处理短序列 patch, 使自注意力机制计算量显著减少, 长序列场景优势显著.

2) 增强局部信息建模: 通过将时间序列分割成 patch, 模型能够更有效地学习每个局部时间段内的模式和依赖关系.

3) 灵活的通道独立性: 模型能够学习每个变量自身的局部特征, 并通过混合层来融合不同变量之间的信息, 从而捕捉变量之间的相互依赖关系.

4) 高效的长距离依赖建模: 分块策略使得 Transformer 的自注意力机制能捕捉长距离的时间依赖关系, 这对于预测仓库活动的长期趋势至关重要.

综上所述, PatchTST 模型通过其独特的 patch 处理方式和通道独立性设计, 在计算效率、局部信息捕获和预测准确性之间取得了良好的平衡, 使其非常适合对具有复杂时间依赖性和潜在长期趋势的仓库活动指标进行预测, 其优异的性能将在后续的实验结果中得到进一步验证.

2.4　仓库活跃度评估方法

为准确评估仓库的长期活跃程度, 本文设计了评估方法 DTA. 具体地, 本文利用 PatchTST 模型预测仓库未来一段时间序列的活动数据, 时间序列的时间步是天, 时间步上的值的现实含义是当日活动指标增量相较于该活动指标的历史单日增量极值的比例, 也就是增量比例. 本文计算得到预测时间范围内各个活动指标的增量比例的均值, 将每个活动指标的增量比例均值通过 Sigmoid 函数映射到同一个数值范围 [0.5, 1), 从而控制大型仓库活动指标的边际效应. 将增量比例与指标权重相乘后再进行累加, 最后进行放缩. 预测范围内活动指标增量比例越高, 活跃度评分越高.

权重设计上, 本文结合层次分析法和 Zhao 等[18] 的研究, 确定各活动指标的权重 (表 1).

本文通过对仓库未来活动指标进行利用, 设计了仓库活跃度评估公式 DTA, 以 S 表示 DTA 分数.

中国知网　https://www.cnki.net

其计算公式为

$$S=\frac{\left(\sum_{i=1}^{n} W_i \operatorname{Sigmoid}(\bar{x})\right)-4.7966}{4.7966}\times 100\%. \tag{2}$$

式 (2) 中: $\bar{x}$ 代表未来 12 个周期的指标均值, n 表示变量数量. $\bar{x}$ 通过 Sigmoid 函数后被映射到范围 [0.5, 1), W_i 表示第 i 个活动指标的权重, 求和部分的取值范围为 [4.796 6, 9.593 2), 最后通过放缩操作将分数转化为百分制.

表 1 仓库活跃指标权重

Tab. 1 Weight of repository activity indicators

活动指标	权重	活动指标	权重
Open issue	2.223 5	PRcomment	0.742 7
Issue comment	0.525 2	MergePR	2.033 9
OpenPR	4.067 9		

为进一步说明 DTA 公式的合理性, 现从数学推导与边界行为两个方面展开分析. 数学推导方面, 公式中引入 Sigmoid 函数对未来活动指标均值 $\bar{x}$ 进行非线性压缩处理, 单个活动指标计算结果的输出范围被严格限制在 [0.5, 1), 同时 Sigmoid 函数二阶导数小于 0, 符合边际递减效应, 可有效抑制极端值影响, 并保持指标的区分度. 该区间设定既避免了低活跃仓库因得分过低导致信息丢失, 也避免了高活动仓库因得分过高造成评价饱和, 具有良好的数值稳定性.

边界条件方面, 当所有指标预测值均接近其历史最大值时, Sigmoid 函数映射后趋近于 1, DTA 分数趋近于上限 9.593 2, 映射至分数接近 100 分, 表示仓库保持着极高活跃状态; 反之, 若预测值长期维持在历史最小值附近, 分数趋近于 4.796 6, 映射至分数接近 0 分, 表示仓库可能进入不活跃阶段. 该设计确保了评估结果在逻辑与数值上的连续性与可解释性. 综上, 该公式有如下优势.

1) 基于未来的新视角. 本文创新性地利用了仓库未来的活动指标数据, 对仓库的活跃程度评估加入了新的视角, 最后得出的仓库活跃分数包含了对其未来发展情况的考量.

2) 公平统一. 得益于数据集处理中最后一步的最大最小值放缩方法, 各个仓库的活动指标此时被放缩到同一个标准. 此时的指标值体现的是该指标相较于其历史最高值和最低值的关系, 而不是指标数量绝对值, 越接近放缩上限表明越接近历史极大值. 时间序列预测模型的输入和输出的值都是这样的比例值, 因此, 本文摆脱了活动指标绝对值数量的影响, 使活跃度分数能够反映仓库的发展趋势.

3 实验结果与分析

3.1 实验参数设置

在时间序列预测任务中, 模型的回看窗口大小与预测时间长度对预测性能有着重要的影响. 因此, 本文利用前人研究结果中仓库参与者的活跃时间与传统的办公时间相关, 工作日的活动时间明显多于周末的结论, 将开发者活动周期定为 7 d, 也就是 1 个工作周. 为了使预测范围能够在一定程度上代表仓库未来的发展趋势, 将预测长度设置为 12 个周期, 也就是 84 d, 近似 3 个月. 同样地, 为了能够准确预测仓库未来 12 个周期的发展, 将回看窗口的长度也设置为 12 个周期. 据调查, 本文是第一个在仓库活动指标时间序列预测上引入开源参与者活动周期的研究, 对周期的利用使本文的预测包含更加全面的信息.

综合实验结果, 为确保模型达到最优预测效果, 其余关键参数设置如表 2 所示.

中国知网 https://www.cnki.net

表 2　时间序列预测模型主要参数设置

Tab. 2　Main parameter settings for time series prediction models

参数	数值	参数	数值
编码器层数	5	缩放因子	3
解码器层数	5	注意力头数量	2

3.2　RQ1: 如何准确地预测仓库的发展趋势

为确定适合开源仓库活动指标预测任务的预测模型, 本文使用均方误差 (Mean Square Error, MSE, 记为 E_{MS}) 和平均绝对误差 (Mean Absolute Error, MAE, 记为 E_{MA}) 作为评价不同模型的评价指标, 二者的计算方式分别为

$$E_{\mathrm{MS}} = \frac{1}{n}\sum_{i=1}^{n}(\widehat{y}_i - y_i)^2, \tag{3}$$

$$E_{\mathrm{MA}} = \frac{1}{n}\sum_{i=1}^{n}|\widehat{y}_i - y_i|. \tag{4}$$

式 (3)—(4) 中: $\widehat{y}_i$ 表示预测值, y_i 表示真实值, n 代表变量数量. E_{MS} 与 E_{MA} 是反映预测值与真实值之间差异程度的一种度量. 二者的值越小, 代表预测误差越小, 预测效果越好. 本文选择了 MSE 作为模型的损失函数, 因为 MSE 的梯度能够跟随 loss 的变化而调整, MAE 的更新梯度则始终相同. 也就是说, 即使对于很小的损失值, 梯度也很大, 这样不利于模型的迭代. 但是, 在后文比较不同模型的预测效果时, 仍然展示了模型的 MSE 与 MAE, 因为相比于 MSE, MAE 可以更加真实地反映真实值和预测值之间的差别, 不会因为平方的关系, 导致误差大于 1 时被放大.

对各模型设置好统一参数进行实验, 不同模型的预测效果结果如表 3 所示. 本文从数据集中依据不同编程语言类型分层抽取了 100 个仓库, 计算模型在这些仓库活动指标预测上的 MSE、MAE 及其均值. PatchTST 模型在 MSE_AVG 上和 MAE_AVG 上均取得了最小值, 优于其他模型, 表明其拥有准确的预测效果. 另外, PatchTST 模型的实际预测效果见图 5, 表明了模型在 Microsoft/Vscode 仓库的 OpenPR 活动指标的预测效果, 可以看到预测值与真实值能够在周期及实际值上有良好的贴合.

表 3　不同时间序列模型在随机抽样仓库集中的预测误差

Tab. 3　Prediction errors of different time series models in randomly sampled repository sets

Model	MSE_AVG	MAE_AVG	Model	MSE_AVG	MAE_AVG
Autoformer	5.096	1.071	TimesNet	5.148	0.980
iTransformer	4.872	0.939	PatchTST	4.660	0.903
NonstationaryTransformer	4.852	0.973			

为了验证数据处理的有效性. 本文分别按照随机种子 1—5 抽取数据集中的 100 个仓库, 对比 PatchTST 模型在这些仓库的原始活动指标数据和清洗后数据的训练效果差异, 具体结果如表 4 所示. 可以看到, 清洗后的数据普遍比未清洗的数据得到更低的预测误差. 接着, 通过计算清洗后数据与原始数据在 MSE 上的相对变化, 评估数据清洗对预测性能的提升效果. 在这 5 份随机样本中, 数据清洗后预测任务的表现平均改善了 47.5%, 表明了数据清洗的有效性.

本节实验内容回答了“如何准确预测仓库的发展趋势”这一问题. 本文发现, 若需要对仓库活动指标进行预测, 需要采取异常值处理和数据平滑方法对数据进行适当清洗以保证预测准确性. 在时间序

中国知网 https://www.cnki.net

列预测模型上, PatchTST 模型比其他模型能更准确地进行预测.

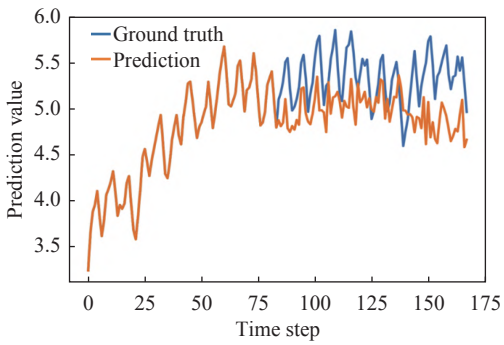


图 5 Microsoft/Vscode 仓库的 OpenPR 活动指标预测效果

Fig. 5 Prediction performance chart of OpenPR activity indicators for microsoft/vscode repository

表 4 随机采样仓库数据集数据清洗前后预测误差对比

Tab. 4 Comparison of prediction errors before and after data cleaning for randomly sampled repository datasets

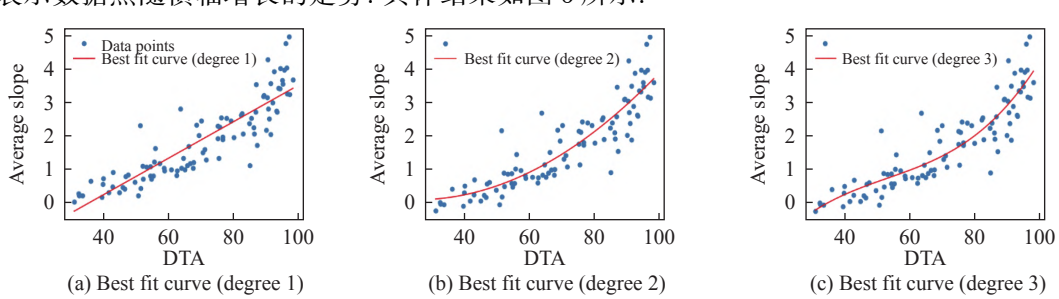
随机种子	MSE		随机种子	MSE	
	原始数据	清洗数据		原始数据	清洗数据
1	102.3	7.8	4	22.9	9.6
2	14.9	5.0	5	7.3	6.4
3	6.0	5.5			

3.3 RQ2: 如何评估一个仓库的活跃度

为验证本文设计的仓库活跃度分数 DTA 的有效性, 本文从仓库长期活跃的现实意义出发进行了实验. 当一个仓库获得高分时, 表明仓库未来的活跃状态良好, 其活动指标数量也应保持较高增长. 因此, 本文选择了活动指标累积增量的平均斜率作为衡量活跃度分数是否有效的指标. 平均斜率是描述函数或曲线变化速率的一个概念, 指的是在某个区间内, 函数变化量与自变量变化量的比值. 它可以用来表示某个量随时间、空间或其他因素的平均变化速度或趋势. 平均斜率 K 的计算公式为

$$K = \frac{Y_n - Y_1}{n-1}. \tag{5}$$

式 (5) 中: n 为序列长度. 设原始的预测值序列为 $\Delta y = \{\Delta y_1, \Delta y_2, \cdots, \Delta y_n\}$, 则 $Y_n = \sum_{i=1}^{n} \Delta y_i$. 可得到累计值序列 $Y = \{Y_1, Y_2, \cdots, Y_n\}$. 本文按照不同类型的编程语言类型进行分层随机抽样, 最后共抽取 100 个仓库. 基于这些仓库计算的分数和平均斜率绘制散点图, 并使用多项式回归模型拟合一条最佳曲线表示数据点随横轴增长的走势. 具体结果如图 6 所示.



(a) Best fit curve (degree 1) (b) Best fit curve (degree 2) (c) Best fit curve (degree 3)

图 6 分别基于一次、二次和三次多项式的最佳拟合曲线

Fig. 6 Optimal fitting curves based on first-order, second-order, and third-order polynomials

中国知网 https://www.cnki.net

图 6(a)—(c) 中, 横坐标为 DTA 分数, 纵坐标为平均斜率, 分别使用不同幂次的函数拟合数据点的最佳函数. 可以看到, 无论是一次还是三次多项式, 最佳拟合函数都满足单调递增的特性, 表明分数与平均斜率为正相关关系.

为了进一步验证本文的活跃度分数与平均斜率的关系, 本文引入了 Spearman 相关系数与 Pearson 相关系数定量地度量两者的相关性, 实验结果如表 5 所示. 同样基于编程语言随机抽样的仓库, 研究了它们的活跃度分数与各个活动指标的平均斜率, 以及各平均斜率的平均值之间的相关性. 可以看到, 活跃度分数与总平均斜率的 Spearman 相关系数和 Pearson 相关系数分别为 0.92 和 0.88, 表明两者具有非常高的单调正相关性. 同时, OpenPR 属性在活动指标中与分数有最强的正相关性, 表明拥有更多 OpenPR 的仓库往往更能够保持活跃.

表 5　平均斜率以及各活动指标与 DTA 的相关性

Tab. 5　The average slope and correlation between various activity indicators and DTA

活动指标	Spearman	Pearson	活动指标	Spearman	Pearson
平均斜率	0.92	0.88	OpenPR	0.85	0.85
Issue comment	0.74	0.74	PRcomment	0.73	0.71
Open issue	0.73	0.72	MergePR	0.81	0.78

本节利用平均斜率验证了本文提出的仓库长期活跃度分数的有效性, 并通过相关系数验证了分数与发展趋势的平均斜率具有强正相关性.

4　结　　论

本文设计了一个新颖的方法评估仓库的长期活跃程度. 该方法创新性地利用仓库的未来发展趋势信息进行活跃度评估, 也首次将开发者活跃周期引入了开源领域的时间序列预测任务. 在获取仓库未来发展趋势的过程中, 本文制作了一份 GitHub 高星仓库活动指标时间序列数据集, 涵盖 40 余种编程语言和 19 种许可证, 总计拥有约 45 万条数据量. 此外, 本文通过实验分析找到了最适合开源软件仓库领域时间序列预测任务的模型: PatchTST. 并利用该模型验证了数据预处理上的有效性, 模型的预测效果平均改善了 47.5%. 对于活跃度公式 DTA, 本文通过活动指标的平均斜率以及分数与平均斜率的相关系数验证了分数的有效性和强正相关性. DTA 的引入能够帮助开源社区参与者识别出未来保持长期活跃的仓库, 确定参与重心, 促进开源社区构建和数字基础设施建设.

尽管本文验证了基于 DTA 方法的有效性, 但仍然存在着不足. 首先, 本文所构建的数据集主要来自 GitHub 平台, 无法完全代表其他开源社区. 其次, 尽管在活动特征选择过程中已从文本交互与代码贡献两个核心维度进行了系统考量, 但鉴于 GitHub 活动指标种类繁多, 仍难以实现全面覆盖. 最后, 在模型选择过程中, 受限于部分模型未公开源码或调研范围的局限, 尚未在数据集上实现更大范围的模型验证, 因此, 关于 PatchTST 为当前开源软件仓库领域最优预测模型的结论仍存在一定局限性. 在未来的工作中, 后续将考虑采用集成学习方法, 综合多模型优势获取更准确的活动指标, 此外, 应通过拓展指标维度, 融合更多细粒度的活动特征, 以提升仓库活跃度评估的全面性与准确性.

5　数据获取

见 https://github.com/Tenth-crew/DTA_dataset/tree/main.

[参　考　文　献]

[1]　STAFF G. Octoverse: AI leads Python to top language as the number of global developers surges [EB/OL]. (2024-10-29)[2025-07-07].

中国知网　https://www.cnki.net

https://github.blog/news-insights/octoverse/octoverse-2024.

[2] DECAN A, MENS T, GROSJEAN P. An empirical comparison of dependency network evolution in seven software packaging ecosystems [J]. Empirical Software Engineering, 2019, 24(1): 381-416.

[3] AIT FONOLLÀ A, IZQUIERDO J L C, SAGRERA J C. An empirical study on the survival rate of GitHub projects [C]// 2022 IEEE/ACM 19th International Conference on Mining Software Repositories (MSR). IEEE, 2022.

[4] TAN X, ZHOU M H, SUN Z Y. A first look at good first issues on GitHub [C]// Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, 2020.

[5] YIN L K, CHEN Z Z, XUAN Q, et al. Sustainability forecasting for Apache incubator projects [C]// Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, 2021.

[6] XIAO W X, HE H, XU W W, et al. How early participation determines long-term sustained activity in GitHub projects? [C]// Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, 2023.

[7] COELHO J, VALENTE M T, MILEN L, et al. Is this GitHub project maintained? Measuring the level of maintenance activity of open-source projects [J]. Information and Software Technology, 2020, 122: 106274.

[8] STĂNCIULESCU Ş, YIN L, FILKOV V. Code, quality, and process metrics in graduated and retired ASFI projects [C]// Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. Singapore: Association for Computing Machinery, 2022: 495–506.

[9] VALIEV M, VASILESCU B, HERBSLEB J. Ecosystem-level determinants of sustained activity in open-source projects: A case study of the PyPI ecosystem [C]// Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, 2018.

[10] ZHANG Y X, ZHOU M H, MOCKUS A, et al. Companies' participation in OSS development–an empirical study of OpenStack [J]. IEEE Transactions on Software Engineering, 2019, 47(10): 2242-2259.

[11] LEHMAN M M. Programs, life cycles, and laws of software evolution [J]. Proceedings of the IEEE, 2005, 68(9): 1060-1076.

[12] XIA X Y, WENG Z J, WANG W, et al. Exploring activity and contributors on GitHub: Who, what, when, and where [C]// 2022 29th Asia-Pacific Software Engineering Conference (APSEC). IEEE, 2022.

[13] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need [EB/OL]. (2023-08-02)[2025-07-07]. https://arxiv.org/pdf/1706.03762.

[14] LI S, JIN X, XUAN Y, et al. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting [EB/OL]. (2020-01-03)[2025-07-07]. https://arxiv.org/pdf/1907.00235.

[15] WU H, XU J, WANG J, et al. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting [J]. Advances in Neural Information Processing Systems, 2021, 34: 22419-22430.

[16] LIU Y, HU T, ZHANG H, et al. iTransformer: Inverted transformers are effective for time series forecasting [EB/OL]. (2020-01-03)[2025-07-07]. https://arxiv.org/pdf/2310.06625.

[17] NIE Y Q, NGUYEN N H, SINTHONG P, et al. A time series is worth 64 words: Long-term forecasting with transformers [EB/OL]. (2023-03-05)[2025-07-07]. https://arxiv.org/pdf/2211.14730.

[18] ZHAO S Y, XIA X Y, FITZGERALD B, et al. OpenRank leaderboard: Motivating open source collaborations through social network evaluation in Alibaba [C]// 2024 IEEE/ACM 46th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP). IEEE, 2024: 346-357.

(责任编辑: 张　晶)

中国知网 https://www.cnki.net